

I. Prompt Engineering と Context Engineering の違い

本質的な違いは、「何を調整対象とするか」と「その目的」にあります。

Prompt Engineering:

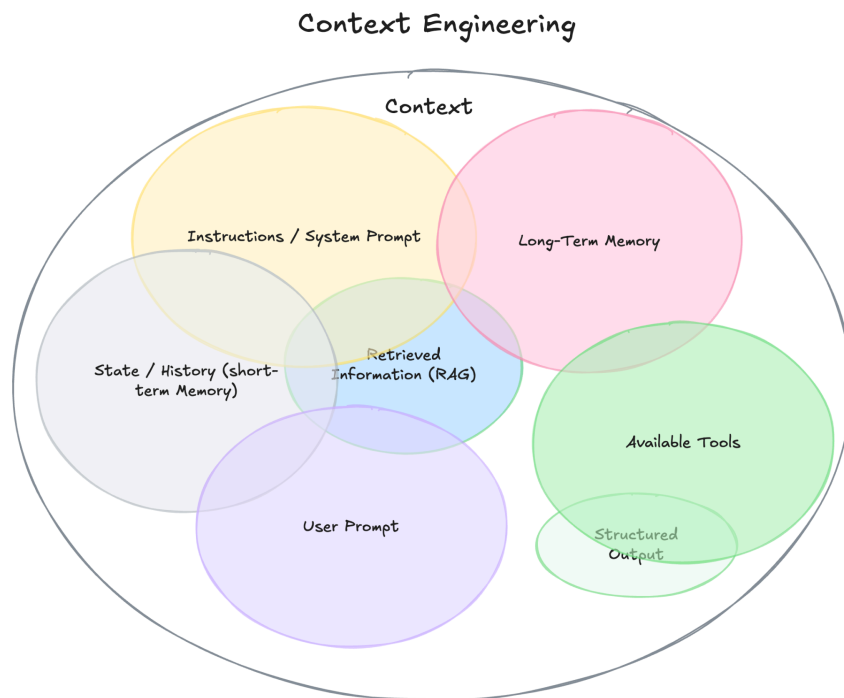
「どのように質問するか」に焦点を当てます。

これは主にエンドユーザーのための手法であり、要求をできるだけ明確に伝えることを目的とします。

Context Engineering:

「何を提供するか」に焦点を当てます。

これは開発者のための専門領域であり、背景情報、検索結果(RAG)、過去の対話履歴などを組み立て、モデルに入力する前に動的なシステムを構築することを指します。



II. Context Engineering の進化 (Hua et al., 2025)

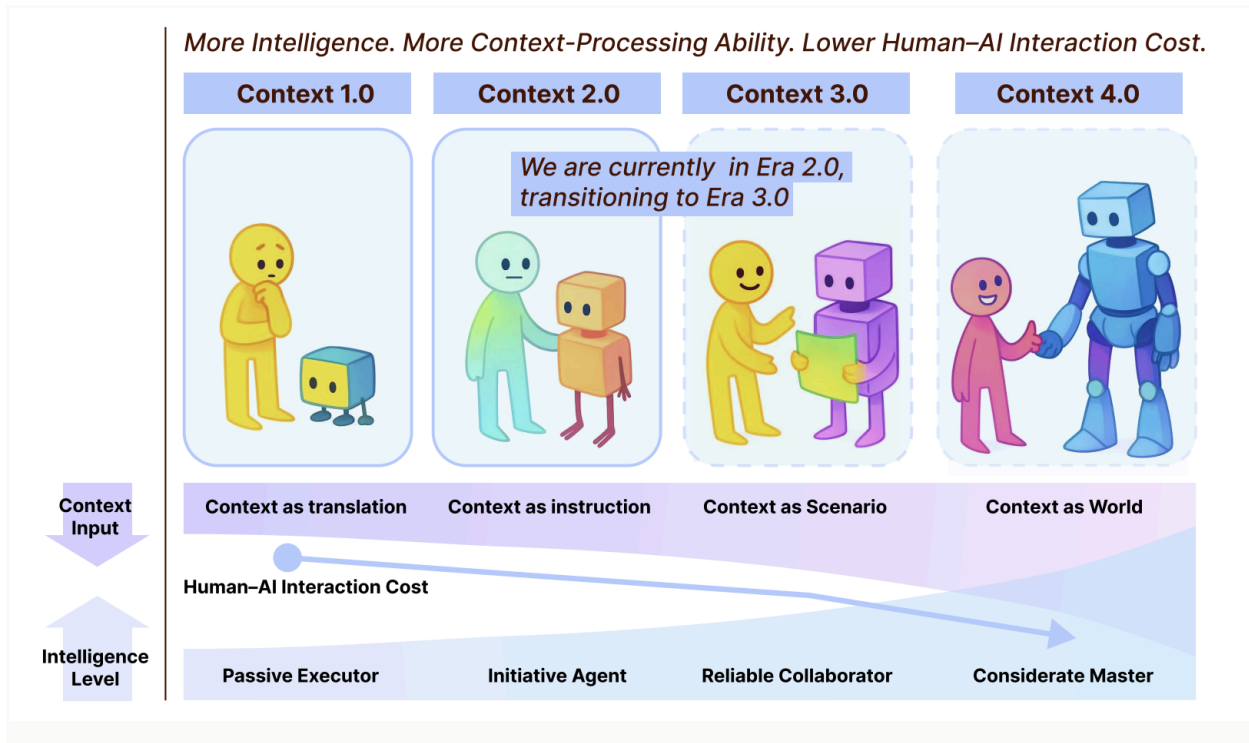
参考資料: <https://arxiv.org/abs/2510.26493>

Hua et al. (2025)によれば、Context Engineering は ChatGPT とともに突然登場したものではありません。

その起源は1990年代にまで遡り、20年以上の歴史があります。

変化したのは、AI の知能レベルと、それに伴って必要とされる「コンテキスト」の性質です。

論文では、進化を4つの段階に分類しています。



Era 1.0 —「Context as Translation」(1990年代)

知能レベル: 初期的なコンピュータ、ルールベースAI

AIの役割: Passive Executor(受動的実行者)

この時代では、コンテキストは完全に構造化され、低エントロピーで、ビット単位まで明確である必要がありました。

コンピュータに何かをさせたい場合、意図を機械が理解できる言語(SQL、XML、厳密なスキーマを持つAPIコールなど)に翻訳しなければなりません。

翻訳コストはすべてエンジニアが負担します。

機械は意図を推測せず、単に実行するだけでした。

Era 2.0 —「Context as Instruction」(現在)

知能レベル: LLM + エージェント

AIの役割: Reliable Collaborator(信頼できる協働者)

これが現在の世界です。

LLMは自然言語を理解し、曖昧さに耐え、不完全な指示から意図を推論できます。

現在のコンテキストには以下が含まれます:

- プロンプト
- システムメッセージ

- 会話履歴
- ツール定義
- RAGチャンク

より自然な形式になっています。

しかし、「信頼できる協働者」とはいえ、AIには依然として明確な指示が必要です。
コンテキストは慎重に設計しなければなりません。
言われていないことをAIが自動的に推論するわけではありません。

Era 3.0 — 「Context as Scenario」(近未来)

知能レベル: 人間レベルAI

AIの役割: Considerate Master

この段階では、AIは詳細な指示を必要としません。
状況(シナリオ)を理解し、人間のように自然に協働します。

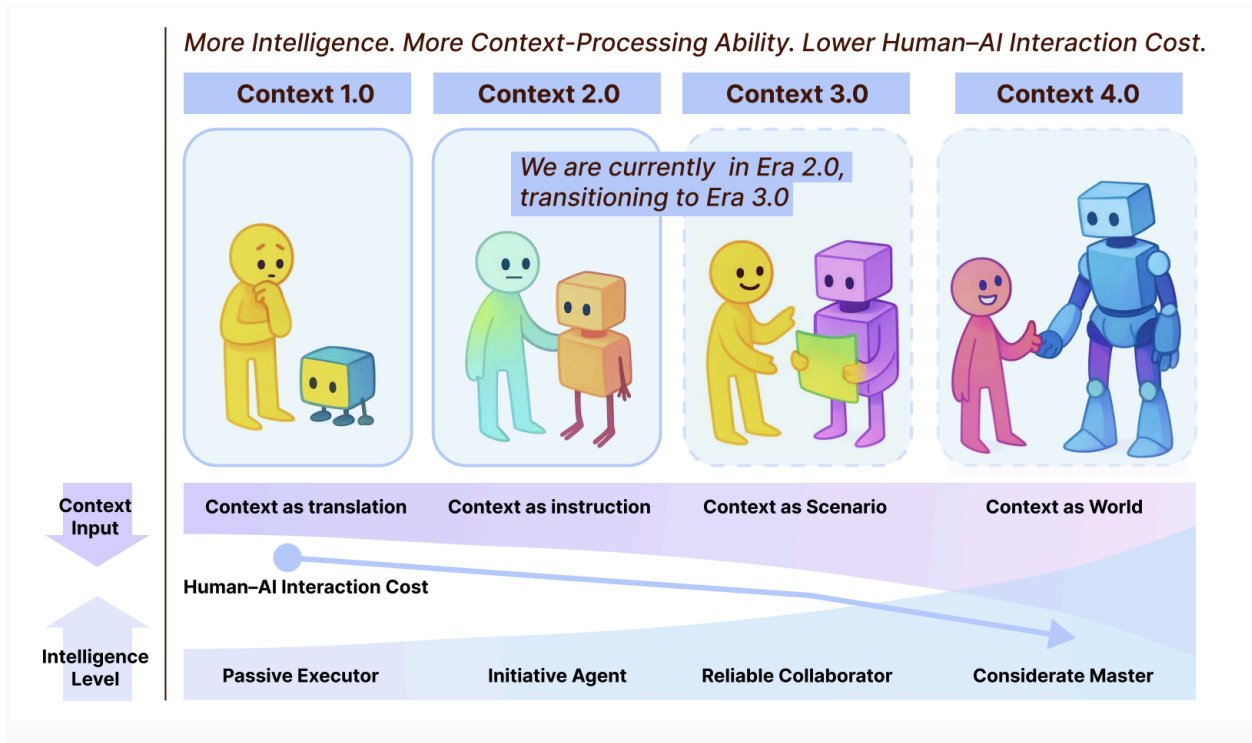
明示的なコンテキストがなくても、優秀な同僚同士が短い言葉で理解し合うように、暗黙的な前提を推測できます。

Era 4.0 — 「Context as World」(遠い未来)

知能レベル: 超人的AI

この段階では、AIはコンテキストを理解するだけでなく、自ら生成します。
ユーザーが言語化していないニーズを察知し、ほぼ完全な自律性で行動します。

ここでのコンテキストとは、AIが相互作用する「世界全体」です。



論文の Figure 1 (知能レベルが上昇し、インタラクションコストが減少する棒グラフ)を見ると、明確なパターンがあります：

- AIが賢くなるほど
- コンテキストはより自然になり
- インタラクションコストは下がり
- ユーザーの負担は減少する

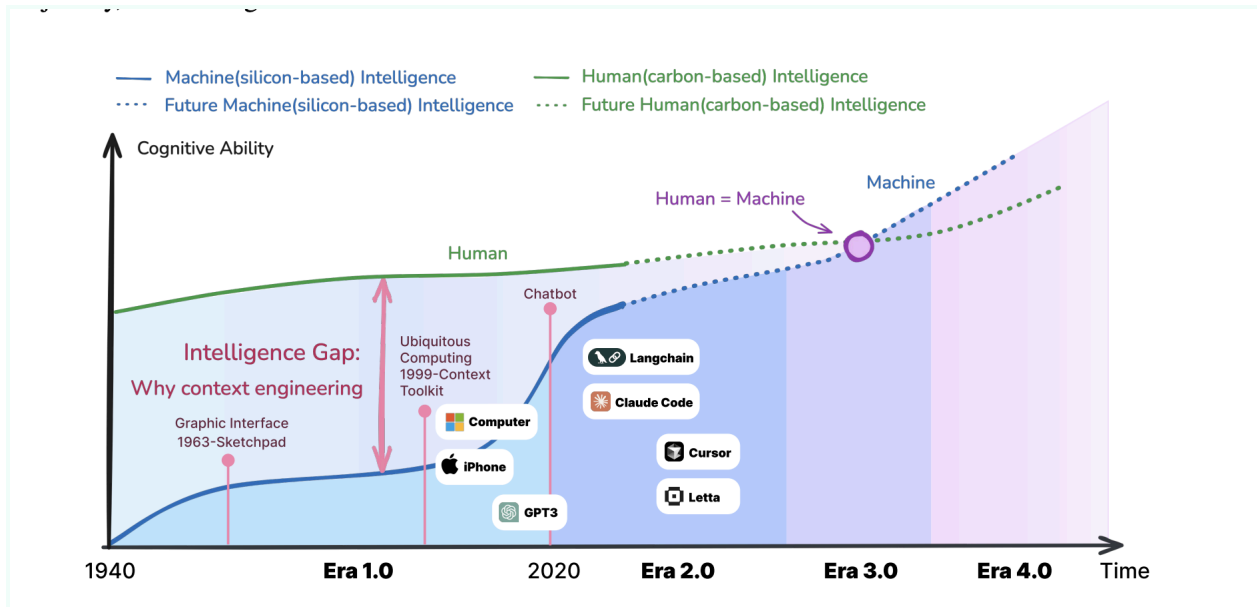


Figure 2 では、人間(炭素ベース)とAI(シリコンベース)の認知能力の軌跡が描かれています。両者のギャップこそが、Context Engineering が存在する動機です。

Context Engineering とは何か？

Context Engineering は、「体系的なエントロピー削減プロセス」と理解できます。

人間の意図は通常：

- 曖昧
- 非構造的
- 解釈の余地が多い

一方、LLMが最もよく機能するのは：

- 構造化されたデータ
- 明確な文脈
- 事前定義された制約

したがってエンジニアの役割は：

高エントロピーな人間の意図を
低エントロピーな表現へ変換すること

III. 現在の実装戦略

AI開発フレームワーク LangChain のブログでは、4つの中核的な実装戦略がまとめられています。

戦略1: Write (書く) — AIに「下書き」と「長期記憶」を与える

AIにも「下書き」や「ログ」が必要です。

複雑なタスクを処理する際、AIが中間段階の思考を一時的な「Scratchpad (下書き領域)」に書き出したり、重要な結論をセッションをまたいで保持される「長期記憶 (Memory)」モジュールに保存できるようにします。

これにより、メインの対話ウィンドウを整理された状態に保ちながら、AIが継続的に学習し、過去を振り返ることが可能になります。

戦略2: Select (選択)

量よりも質が重要です。

AIがタスクに直面した際、RAGなどの技術を活用し、広範な知識ベース(ドキュメント、ツール、メモリなど)から現在最も関連性の高い情報のみを選択する「インテリジェント・オーケストレーター」を起動し、それをモデルに提供します。

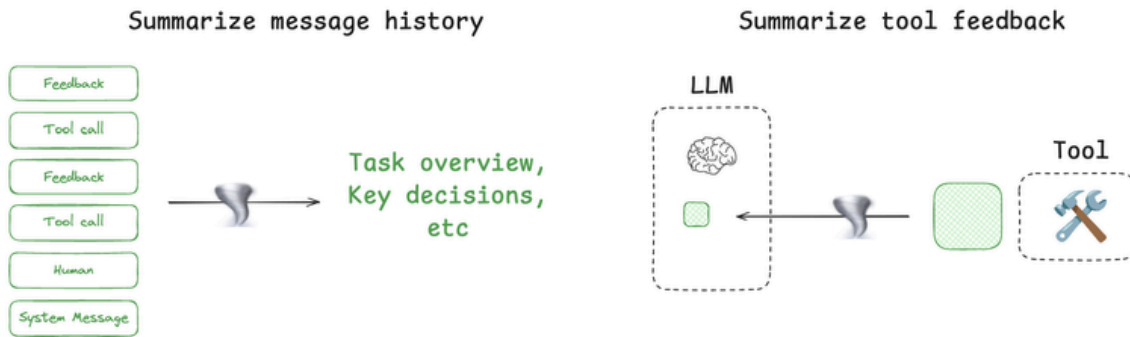
これにより、情報過多を防ぎ、モデルが本質的な問題に集中できるようになります。

戦略3: Compress (圧縮)

コンテキストメモリには限りがあり、あらゆる空間が貴重です。

対話履歴や検索結果の文書が長くなりすぎた場合、「単純化」する方法を学ぶ必要があります。

要約やトリミングといった戦略を通じて、重要な情報を失うことなく、最も本質的な内容だけを貴重なコンテキストウィンドウ内に保持します。

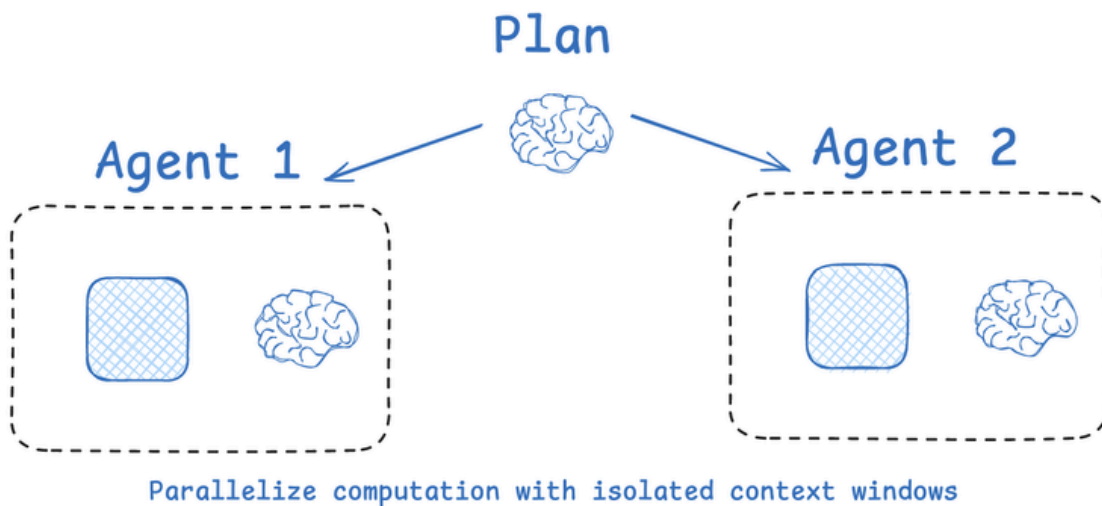


戦略4: Isolate (分離)

分割統治によって複雑さを単純化します。
これはマルチエージェント(Multi-agent)システムの本質です。

あらゆる複雑さに対応する「万能エージェント」に任せるのではなく、大きなタスクを分解し、それぞれ最適化された独立コンテキストを持つ複数の「専門エージェント」に分配します。

各エージェントが自身の役割を果たし、最終的に結果を統合することで、 $1+1>2$ の効果を実現します。



IV 3つの設計柱(論文より)

論文よりは、Context Engineering の構成要素を3つの柱に整理しています。

1. Context Collection & Storage (収集と保存)

- どこから収集するのか？
- どの形式で保存するのか？
- ライフサイクルはどれくらいか？

これは基盤です。コンテキストに投入する情報がなければ、その後のすべては意味を持ちません。

2. Context Management (管理)

- テキスト／マルチモーダル処理
- 階層型メモリアーキテクチャ(ワーキングメモリ、長期記憶、エピソード記憶など)
- コンテキスト分離(マルチエージェント環境でAgent AのコンテキストがAgent Bに「汚染」されないようにする)

3. Context Usage (活用)

- システム内外でのコンテキスト共有
- Proactive Inference (AIが必要なコンテキストを能動的に推論)
- Lifelong Preservation (単一セッションではなく、エージェントのライフサイクル全体にわたる保持)

V 現在実際に機能しているもの

実運用システムからの実践パターン

1. Gemini CLI のアプローチ

Gemini CLI は、ファイル構造を活用してAIの世界を定義し、ファイルシステムを一種の知識データベースとして利用します。

仕組み

- GEMINI.md ファイルを使用してプロジェクトのコンテキストを定義
- 階層型ファイル構造 (home → project → subdir) で整理
- 起動時に静的コンテキストをロード
- 対話履歴などの動的コンテキストを蓄積
- 定期的に要約を実施し、空間を最適化

優れている点

- ファイルシステムをデータベースとして扱うことで管理が極めてシンプル
- コンテキスト継承 (子ディレクトリが親のコンテキストを自動的に継承)
- 人間が直接修正可能 (.md ファイルを編集して AI の誤りを修正できる)

不足している点

- 余分なコンテキストを自動的に削除する仕組みがない
- ユーザーによる手動管理が必要
- 定期更新しないとコンテキストが陳腐化する可能性がある

VI ビジョン: コンテキストのためのセマンティックOS

コンテキストを「第一級市民」として扱う

従来のOSにおけるRAM管理のように、コンテキストを補助データではなく、独立した最重要エンティティとして扱います。

能動的ガバナンス

システムは受動的に保存するだけでなく、重要度や時間に基づいて知識を追加・修正・削除(忘却)します。

人間の記憶モデルの模倣

ワーキングメモリと長期記憶からなる階層型アーキテクチャを採用し、即時検索と長期保存を最適化します。

推論チェーンの追跡

AIのすべての結論は、思考プロセスを透明化するための証拠チェーン(reasoning chains)と紐付けられます。

自己修正機能

最終回答を出す前に、論理的誤りやデータの矛盾を検出し、自動修正する能力を持ちます。

必要な構成要素

1. 大規模セマンティックストレージ

- 効率的なベクトル保存: 意味ベース検索を可能にするベクトルデータベースの活用
- グラフベース関係: 知識グラフとして記憶を整理し、複雑な関係性を把握
- 意味を失わない圧縮: 巨大データを本質を保持したまま「意味ベクトル」に圧縮
- 階層的処理: 詳細から抽象まで多層構造で理解

2. 説明可能な推論 (Explainable Reasoning)

- すべての決定の追跡
- 使用されたコンテキストの可視化
- ユーザーによる修正介入
- 監査ログ (audit trail) の維持

3. Lifelong Context (生涯コンテキスト)

- 数年単位で保存される永続的コンテキスト
- ユーザーとともに進化
- 重要情報を決して忘れない (意味保存原則)
- プライバシー保護 (安全領域やローカル暗号化による保護)

必要であれば、より学術的な文体、ビジネス向け要約版、あるいはプレゼンテーション用に再構成したバージョンも作成できます。